AI Usage Log - Sprint 1

SOEN 341 Software Process | CareerConnect | Fall 2026

Team member	Sayf [add surname]
Project	CareerConnect - Job Search and Application Tracking Platform
GitHub repository	https://github.com/Sayf-01/SOEN341-2026
Sprint	Sprint 1
AI interactions logged	3 (see summary below). Full prompts and responses are in the appendices.

Summary of entries

Task	Title	Purpose	Decision
1.1	Generate 10 AI user stories	Brainstorming / requirements	Accepted
1.2	Test database connection and merge steps	Testing / debugging / Git guidance	Modified before use
1.3	Sprint 1 planning document	Planning / documentation	Accepted

Log entries

Task 1.1: Generate 10 AI user stories	
Purpose of AI Use	Brainstorming and requirements elicitation. This is the AI-generated batch of 10 user stories required by the Sprint 1 instructions.
AI tool	[add AI tool name]
Chat Link or Prompt/Response	No chat link available. Full prompt and response are in Appendix A.1.
AI-Suggested Content	10 user stories. Job seeker (5): upload/update resume; track application status (Applied, Interview, Offered, Rejected); notifications when a job is about to close; dashboard of past and current applications; AI-generated resume feedback. Recruiter (5): create/publish job postings; edit or close a posting; view all applicants for a posting; update an applicant's status; search and filter candidate profiles by skills or experience.
Validation	Compared each story with the feature list in the project overview. All 10 map to listed features (resume management, job posting management, status tracking, application history dashboard, deadline notifications, AI resume feedback). The 10 AI stories are kept separate from the 5 team-written stories.
Decision	Accepted. Used as the AI-generated batch of 10; the team's own stories are documented separately under "Team-Generated User Stories and Features".
Reflection	The AI covered gaps in our own stories, especially the recruiter side and status tracking. The stories are fairly generic, so the original ideas had to come from the team's own brainstorming.

Responsible Person	Sayf

Task 1.2: Test database connection code and steps to merge	
Purpose of AI Use	Testing, debugging, and Git guidance for the backend database setup (SQLAlchemy + Neon PostgreSQL).
AI tool	[add AI tool name]
Chat Link or Prompt/Response	Chat link: https://chatgpt.com/share/6ab9b5d9-0fe8-83ea-abc9-64c0d3adb21c (labelled "CHECKING DATABASE"). Key prompts and responses are also in Appendix A.2.
AI-Suggested Content	(1) A temporary script, test_db.py, that imports the engine, Base and User model and runs Base.metadata.create_all() to create the users table in the Neon database. (2) When the script failed with an ImportError, a suggested cause (App vs app capitalization and a possibly unsaved connection.py). (3) When it then failed with ModuleNotFoundError: No module named 'psycopg', the fix pip install "psycopg[binary]" and re-freezing requirements.txt. (4) Steps to delete the test file, check git status, commit, push feature/setup-database, and open and merge a pull request into main.
Validation	Ran test_db.py in the project virtual environment against the real Neon database. The first runs failed with the errors above; after installing the driver, the script ran and printed the success message. Result confirmed by running the code, not just reading the AI's answer.
Decision	Modified before use. The script's imports were adjusted to match the project's App folder name, the extra database driver was installed, and requirements.txt was regenerated. test_db.py was a temporary file and not part of the committed code.
Reflection	The AI said the code was ready before anything had been tested, and its first explanation of the error was a guess. Reading the actual error messages is what led to the real problem (missing driver). Lesson: always run AI-reviewed code instead of trusting the review.
Responsible Person	Sayf

Task 1.3: Sprint 1 planning document	
Purpose of AI Use	Planning and documentation: turning rough sprint-planning notes into a properly formatted Sprint 1 Plan, and calculating team capacity.
AI tool	[add AI tool name]
Chat Link or Prompt/Response	No chat link available. Full prompt (my notes) and the AI's summary are in Appendix A.3.
AI-Suggested Content	A structured Sprint 1 Plan document built from my notes (repo setup, workflow and branching, risks, effort estimation, team capacity, user stories, demo). The AI also did the capacity math: 6 people x 4 hrs/week x 2 weeks = 48 hours, about 10 hours for ramp-up, about 38 hours for the actual work.

Validation	Re-checked the arithmetic by hand: 6 x 4 x 2 = 48; 20% of 48 is 9.6, rounded to 10; 48 - 10 = 38. The workflow, risks, and estimation approach came from the team's own notes; the AI only formatted them.
Decision	Accepted.
Reflection	Useful for structuring messy notes and for the capacity calculation. The decisions themselves (GitHub Flow, story points, risks) were ours, so the AI saved formatting time rather than changing the plan.
Responsible Person	Sayf

Appendix A - Prompts and AI Responses

Text below is copied from the original conversations. Line breaks were restored where the export had flattened them. Emoji in the AI's code output were removed because they do not render in this PDF.

A.1 Task 1.1 - User stories

Prompt

Here are my 5 user stories:
1. [Job seeker] provides career goals and their CV so the system can match them with the most appropriate jobs (AI).
2. Filter job postings (by location, title, or category) so that I can quickly find roles relevant to me.
3. As a job seeker, I want to save jobs to a favourites list so that I can revisit postings I'm interested in later.
4. As a job seeker, I want to create an account and log in to access my profile and view application history.
5. Recruiters can view my qualifications when I apply to jobs.

Create me 10 more users stories

AI response (user stories)

1. As a job seeker, I want to upload and update my resume so that recruiters can view my latest qualifications.
2. As a job seeker, I want to track the status of each application (Applied, Interview, Offered, Rejected) so that I can stay organized throughout my job search.
3. As a job seeker, I want to receive notifications when a job I'm interested in is about to close so that I don't miss application deadlines.
4. As a job seeker, I want to view a dashboard summarizing all my past and current applications so that I can quickly assess my progress.
5. As a job seeker, I want to receive AI-generated feedback on my resume so that I can improve it before applying.
6. As a recruiter, I want to create and publish job postings so that job seekers can discover and apply to open roles.
7. As a recruiter, I want to edit or close a job posting so that I can keep listings accurate and up to date.
8. As a recruiter, I want to view a list of all applicants for a specific job posting so that I can review candidates efficiently.
9. As a recruiter, I want to update an applicant's status (e.g., move them to Interview or Rejected) so that job seekers are kept informed of their progress.
10. As a recruiter, I want to search and filter candidate profiles by skills or experience so that I can identify strong matches for my open roles.

The AI also reminded me to label AI-generated and team-written stories separately in the submission.

A.2 Task 1.2 - Database test and merge

Prompt 1

okay lets test it my code first and if all is good, pls show me the steps to safely merge it to main. Here is all of my code: base.py, connection.py, __init__.py, user.py (shown below). **Prompt 2:** show me step by step

```
# base.py
from sqlalchemy.orm import declarative_base
Base = declarative_base()

# connection.py
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")
if DATABASE_URL and DATABASE_URL.startswith("postgres://"):
    DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
def get_db():
```

```
    """Creates a temporary database session for a single API request,
    and automatically closes it when the request is done."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# __init__.py
# When we use alembic, it will look for the models in this file.
from App.models.user import User

# user.py
from sqlalchemy import Column, Integer, String, DateTime
from datetime import datetime, timezone
from App.database.base import Base
class User(Base):
    """The 'users' table: stores credentials for registration and login."""
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True, nullable=False)
    email = Column(String(100), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)  # hashed, never plaintext
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))
```

AI response (relevant parts)

Test script suggested by the AI (final version, after the import fix):

```
import sys, os
sys.path.append(os.path.abspath(os.path.dirname(__file__)))
from App.database.connection import engine
from App.database.base import Base
from App.models.user import User

print("Attempting to connect to your Neon Cloud Database...")
try:
    Base.metadata.create_all(bind=engine)
    print("Success! Your 'users' table has been created in your cloud database.")
except Exception as e:
    print("Connection Failed! Error details below:")
    print(e)
```

Run with: `python test_db.py`

Error 1 reported back: ImportError: cannot import name 'engine' from 'App.database.connection'. **AI suggestion:** use the capital-A App in the imports and make sure connection.py is saved and contains the engine line.

Error 2 reported back: ModuleNotFoundError: No module named 'psycopg'. **AI suggestion:** install the driver and update the requirements list:

```
pip install "psycopg[binary]"
pip freeze > requirements.txt
python test_db.py
```

My reply: it worked. **AI's merge steps:**

```
rm test_db.py
git status          # check .env and .venv/ are NOT listed
git add App/ requirements.txt .gitignore
git commit -m "Initialize backend layout, configure SQLAlchemy, and deploy 3NF User model"
git push origin feature/setup-database
# then on GitHub: Compare & pull request -> Create pull request -> (teammate review) -> Merge pull request -> Confirm
```

A.3 Task 1.3 - Sprint 1 planning document

Prompt

[My raw notes] Sprint 1 plan. Init stuff (order matters): github repo, readme, desc of project, tech we using, who's on team + roles, folder for ai logs each person, meeting notes file. Planning section: workflow and branching - GitHub Flow, main branch always works, people branch off for features like feature/registration, PRs need 1 approval before merging; reviewer checks it does what the issue says, isn't messy, doesn't break stuff; a story is

ready when it has criteria + tasks + someone assigned, done when merged + tested + issue closed. Risks: team doesn't know FastAPI/React well (give time to learn, pair people up); work might not be spread evenly (use issues/labels); might try to add too much (stick to the 2 required features); merge conflicts (commit small and often). Effort estimation: story points, Fibonacci (1,2,3,5,8), discuss as a team; 1 = static page, 5 = big thing like auth touching db+api+ui; size all 15 stories once written. Team capacity: 6 people, ~4 hrs a week each, sprint is 2 weeks [need to calc this], ~20% for ramp-up, rest to the 2 features + docs. User stories: min 15 using GitHub issues, tasks assigned; 10 from asking AI, other 5+ our own and kept separate. Demo: registration and login.

Here is my sprint planning, make me a document in a proper format

AI response (summary)

The AI created a formatted "Sprint 1 Plan" document from the notes and filled in the capacity calculation: 6 people x 4 hrs/week x 2 weeks = 48 total hours, with about 10 hours set aside for ramp-up and about 38 hours left for the actual work.

AI Usage Log - Sprint 2 additions

SOEN 341 Software Process | CareerConnect | Fall 2026

Team member	Sayf [add surname]
GitHub repository	https://github.com/Sayf-01/SOEN341-2026
Sprint	Sprint 2 (Oct 8 to Oct 26, 2026)
AI interactions logged	4 Claude chats (Tasks 2.1 to 2.4). These entries continue the Sprint 1 log above, which is unchanged.

Summary of entries

Task	Title	Purpose	Decision
2.1	Prompt writing for the repo AI (SA-9 tests), chat 1	Testing / prompt writing	[confirm]
2.2	Prompt writing for the repo AI (SA-9 tests), chat 2	Testing / prompt writing	[confirm]
2.3	Prompt writing for the repo AI (SA-9 tests), chat 3	Testing / prompt writing	[confirm]
2.4	Final resume and apply tests, push fix and Sprint 2 logs	Testing / Git guidance / documentation	Modified before use

Log entries

Task 2.1: Prompt writing for the repo AI (SA-9 automated tests), chat 1	
Purpose of AI Use	Prompt writing for a second AI that works inside the local repository, to write and verify automated tests for the Sprint 2 endpoints (SA-9), and plain-English explanation of that AI's output. Area covered in this chat: [confirm: profile / resume / apply / other].
AI tool	Claude (Anthropic), used in claude.ai. The generated prompts were run by a second AI inside the local repository.
Chat Link or Prompt/Response	Chat link: https://claude.ai/share/254e4373-1119-4a4e-af3d-95f3f4d797f6
AI-Suggested Content	Step-by-step prompts for the repo AI (absolute rules, short context, exact git and pytest commands, a break-on-purpose check, and a final report format), plus plain-English explanations and reviews of the repo AI's reports.
Validation	Every test or code change was run, not just read. The second AI (the repo AI) had to show the output of every command. Checks used: the full pytest suite had to pass; a break-on-purpose check (one rule temporarily weakened in App/, the matching test had to fail, then App/ restored with an empty git diff); and git status and the branch name had to be clean at the end. The final report was also checked for completeness and for coverage claims without a named test.

Decision	[confirm: Accepted / Modified before use]
Reflection	The repo AI sometimes stops early, writes a pull request description for the wrong thing, renumbers scenario tables, or claims coverage without proof, so each report was checked line by line instead of trusted. Prompts with exact commands and fixed rules gave much more reliable results than open requests.
Responsible Person	Sayf

Task 2.2: Prompt writing for the repo AI (SA-9 automated tests), chat 2	
Purpose of AI Use	Prompt writing for a second AI that works inside the local repository, to write and verify automated tests for the Sprint 2 endpoints (SA-9), and plain-English explanation of that AI's output. Area covered in this chat: [confirm: profile / resume / apply / other].
AI tool	Claude (Anthropic), used in claude.ai. The generated prompts were run by a second AI inside the local repository.
Chat Link or Prompt/Response	Chat link: https://claude.ai/share/5d637a6d-1557-4cd5-897b-8ca8e535c299
AI-Suggested Content	Step-by-step prompts for the repo AI (absolute rules, short context, exact git and pytest commands, a break-on-purpose check, and a final report format), plus plain-English explanations and reviews of the repo AI's reports.
Validation	Every test or code change was run, not just read. The second AI (the repo AI) had to show the output of every command. Checks used: the full pytest suite had to pass; a break-on-purpose check (one rule temporarily weakened in App/, the matching test had to fail, then App/ restored with an empty git diff); and git status and the branch name had to be clean at the end. The final report was also checked for completeness and for coverage claims without a named test.
Decision	[confirm: Accepted / Modified before use]
Reflection	The repo AI sometimes stops early, writes a pull request description for the wrong thing, renumbers scenario tables, or claims coverage without proof, so each report was checked line by line instead of trusted. Prompts with exact commands and fixed rules gave much more reliable results than open requests.
Responsible Person	Sayf

Task 2.3: Prompt writing for the repo AI (SA-9 automated tests), chat 3	
Purpose of AI Use	Prompt writing for a second AI that works inside the local repository, to write and verify automated tests for the Sprint 2 endpoints (SA-9), and plain-English explanation of that AI's output. Area covered in this chat: [confirm: profile / resume / apply / other].
AI tool	Claude (Anthropic), used in claude.ai. The generated prompts were run by a second AI inside the local repository.
Chat Link or Prompt/Response	Chat link: https://claude.ai/share/f6f9da95-1d3f-4901-9efe-541a5eaa6e54

AI-Suggested Content	Step-by-step prompts for the repo AI (absolute rules, short context, exact git and pytest commands, a break-on-purpose check, and a final report format), plus plain-English explanations and reviews of the repo AI's reports.
Validation	Every test or code change was run, not just read. The second AI (the repo AI) had to show the output of every command. Checks used: the full pytest suite had to pass; a break-on-purpose check (one rule temporarily weakened in App/, the matching test had to fail, then App/ restored with an empty git diff); and git status and the branch name had to be clean at the end. The final report was also checked for completeness and for coverage claims without a named test.
Decision	[confirm: Accepted / Modified before use]
Reflection	The repo AI sometimes stops early, writes a pull request description for the wrong thing, renumbers scenario tables, or claims coverage without proof, so each report was checked line by line instead of trusted. Prompts with exact commands and fixed rules gave much more reliable results than open requests.
Responsible Person	Sayf

Task 2.4: Final resume and apply tests, push fix and Sprint 2 logs (Oct 26)	
Purpose of AI Use	Prompt writing for the repo AI to finish and verify the tests for Lamees's resume and apply endpoints (SA-9, LA-2, LA-3), review of the repo AI's reports, fixing a rejected git push, and preparing the contribution log and AI log on the Sprint 2 deadline day (Oct 26, 2026).
AI tool	Claude (Anthropic), used in claude.ai. The generated prompts were run by a second AI inside the local repository (its commits carry a GitHub Copilot co-author line).
Chat Link or Prompt/Response	Chat link: https://claude.ai/share/f41c8949-b5e6-4e35-b442-a6a64bf61066
AI-Suggested Content	(1) A prompt to add the optional cover_note field (max 1000 characters) to the apply endpoint with 4 tests; it was not run because Lamees added the change herself. (2) A corrected version of an existing prompt for the last apply tests: the step for an unmerged branch was replaced (her pull request #104 was already merged), the throwaway-branch rules were removed, and the pull request description was widened to cover both test files. (3) A one-shot prompt covering apply scenarios L1 to L6 and resume scenarios R1 to R8. (4) An explanation of the rejected push (non-fast-forward) and a prompt to merge the remote commit and push without any force option. (5) A comparison of Lamees's written task guide with the real code (400 for automatic validation errors, not 422; the 1000-character cover_note limit comes from the team, not from her guide). (6) Drafts of this contribution log and AI log update.

Validation	The repo AI's report was checked, not trusted. Result: 16 new tests (10 in tests/test_applications_extra.py, 6 in tests/test_resumes_extra.py); full suite 96 passed (80 before). Two break-on-purpose checks: with the closed-job check disabled the new test failed (expected 422, got 201); with the resume owner filter removed the new test failed (expected 404, got 200). After each check App/ was restored, git diff -- App/ was empty and the suite passed again. The first push was rejected because the remote branch had one extra commit (6f8020e). It was merged with a normal merge (merge commit 9b1fb7b), the suite still passed (96), the push succeeded and git status was empty. The report was also checked for completeness, for coverage claims that named a real test, and for a pull request description that covered both test files.
Decision	Modified before use. The prompts were edited before running, and the cover_note prompt was dropped because Lamees made that change.
Reflection	The repo AI stopped at the push error and skipped its final git status check, and its first report did not explain the extra remote commit, so a follow-up prompt was needed. A normal merge, never a force push, was the safe fix. Lamees's written guide did not match the real code on validation status codes, which shows why tests must be written from the real code. The AI is useful for exact prompts and for explaining output in plain English, but every claim still has to be backed by a command that was actually run.
Responsible Person	Sayf